

Documentation du projet Triangulator

Table des matières

1. [Introduction](#1-introduction)
2. [Installation et configuration](#2-installation-et-configuration)
3. [Architecture du projet](#3-architecture-du-projet)
4. [Commandes Make](#4-commandes-make)
5. [Tests](#5-tests)
6. [Couverture de code](#6-couverture-de-code)
7. [Qualité du code](#7-qualité-du-code)
8. [Documentation automatique](#8-documentation-automatique)
9. [Algorithme de triangulation](#9-algorithme-de-triangulation)
10. [Format binaire](#10-format-binaire)
11. [API REST](#11-api-rest)
12. [Dépannage](#12-dépannage)

1. Introduction

Le projet **Triangulator** est un microservice Flask qui effectue la triangulation de Delaunay d'ensembles de points 2D. Il communique avec un service externe **PointSetManager** pour récupérer les ensembles de points, effectue la triangulation, et retourne les résultats au format binaire.

Caractéristiques principales

- **Algorithme de Delaunay (Bowyer-Watson)** : Triangulation optimale qui maximise l'angle minimal
- **Format binaire compact** : Représentation efficace des points et triangles
- **API REST** : Interface HTTP pour la triangulation
- **Visualisation** : Module de génération d'animations et de vidéos du processus de triangulation
- **Tests complets** : 82 tests (unitaires, intégration, API, questionnaires d'erreurs, performance)
- **Couverture excellente** : 100% de couverture de code (modules principaux)
- **Qualité assurée** : Conformité PEP8 avec ruff

2. Installation et configuration

2.1 Prérequis

- Python 3.10 ou supérieur
- pip (gestionnaire de packages Python)
- make (pour l'exécution des commandes)

2.2 Installation des dépendances

```
# Se placer à la racine du projet
cd /Users/macbookair/techniques_de_test_2025_2026

# Créer un environnement virtuel (si pas déjà fait)
python3 -m venv .venv

# Activer l'environnement virtuel
source .venv/bin/activate

# Installer les dépendances de production
pip install -r requirements.txt

# Installer les dépendances de développement (pour les tests)
pip install -r dev_requirements.txt
```

2.3 Structure des dépendances

requirements.txt (production) :

- **flask** : Framework web pour l'API REST
- **requests** : Client HTTP pour communiquer avec PointSetManager
- **matplotlib** : Génération de graphiques et visualisations
- **imageio** : Création de vidéos à partir d'images
- **Pillow** : Manipulation d'images

dev_requirements.txt (développement) :

- **pytest** : Framework de test
- **pytest-requests-mock** : Mocking des requêtes HTTP
- **coverage** : Mesure de couverture de code
- **ruff** : Linter et formateur de code
- **pdoc3** : Génération de documentation HTML
- **markdown** : Conversion de Markdown
- **weasyprint** : Génération de PDF à partir de HTML

2.4 Configuration de l'environnement

```
# Se placer dans le dossier TP
cd TP

# Exporter le PYTHONPATH (déjà configuré dans le Makefile)
export PYTHONPATH=$(pwd)/..
```

3. Architecture du projet

3.1 Structure des fichiers

```

techniques_de_test_2025_2026/
■■■■ .venv/                # Environnement virtuel Python
■■■■ requirements.txt       # Dépendances de production
■■■■ dev_requirements.txt   # Dépendances de développement
■■■■ TP/
■■■■ Makefile              # Commandes d'exécution
■■■■ .coveragerc           # Configuration de la couverture de code
■■■■ PLAN.md               # Plan de tests initial
■■■■ RETEX.md              # Retour d'expérience
■■■■ SUJET.md              # Sujet du projet
■■■■ pyproject.toml        # Configuration ruff et pytest
■■■■ triangulator/         # Code source
■   ■■■■ __init__.py
■   ■■■■ api.py            # API Flask
■   ■■■■ core.py           # Algorithme de triangulation
■   ■■■■ utils.py          # Utilitaires
■   ■■■■ visualizer.py     # Visualisation et animation
■■■■ tests/                # Tests
■   ■■■■ test_unit.py      # Tests unitaires (44 tests)
■   ■■■■ test_performance.py # Tests de performance (6 tests)
■   ■■■■ test_api.py       # Tests API (3 tests)
■   ■■■■ test_integration.py # Tests d'intégration (7 tests)
■   ■■■■ test_error_handlers.py # Tests gestionnaires d'erreurs (22 tests)
■■■■ examples/             # Exemples d'utilisation
■   ■■■■ README.md         # Guide des exemples
■   ■■■■ visualize_example.py # Exemples de visualisation
■■■■ docs/                 # Documentation générée
■   ■■■■ triangulator/     # Documentation HTML
■   ■■■■ DOCUMENTATION.md  # Ce fichier
■   ■■■■ documentation.pdf  # Documentation PDF

```

3.2 Modules principaux

*****triangulator/core.py*****

Contient la logique de triangulation et les conversions binaires :

- **Point** : Classe représentant un point 2D
- **Triangle** : Classe représentant un triangle
- **triangulate(points)** : Algorithme de Delaunay (Bowyer-Watson)
- **pointset_to_bytes(points)** : Conversion points → binaire
- **bytes_to_pointset(binary)** : Conversion binaire → points
- **triangles_to_bytes(points, triangles)** : Conversion triangles → binaire
- **bytes_to_triangles(binary)** : Conversion binaire → triangles

*****triangulator/api.py*****

API Flask exposant l'endpoint de triangulation :

- **GET /triangulation/** : Triangule un ensemble de points

*****triangulator/utils.py*****

Fonctions utilitaires pour la validation et la conversion de données.

*****triangulator/visualizer.py*****

Module de visualisation de l'algorithme de triangulation :

- **visualize_delaunay_step()** : Génère une image d'une étape de triangulation
- **create_delaunay_animation()** : Crée une animation complète de la triangulation
- Génération de vidéos MP4 à partir des frames
- Support de visualisation des cercles circonscrits, bad triangles, polygones

4. Commandes Make

Le fichier **Makefile** fournit des commandes standardisées pour toutes les opérations courantes.

4.1 Commande : `make test`

Description : Lance TOUS les tests (unitaires, intégration, API, et performance)

Utilisation :

```
make test
```

Détails :

```
# Commande exécutée en arrière-plan :  
../.venv/bin/pytest tests/ -v
```

Sortie attendue :

```
===== test session starts =====  
collected 82 items  
  
tests/test_api.py::test_triangulation_success PASSED [ 1%]  
tests/test_api.py::test_triangulation_not_found PASSED [ 2%]  
tests/test_api.py::test_triangulation_bad_request PASSED [ 3%]  
tests/test_error_handlers.py::test_invalid_uuid PASSED [ 4%]  
...  
tests/test_unit.py::test_performance_round_trip_conversion PASSED [100%]  
  
===== 82 passed, 6 warnings in 0.35s =====
```

Quand l'utiliser :

- Avant de committer du code
- Pour vérifier que tout fonctionne après une modification
- En intégration continue (CI/CD)

4.2 Commande : `make unit\_test`

Description : Lance uniquement les tests unitaires et d'intégration (SANS les tests de performance)

Utilisation :

```
make unit_test
```

Détails :

```
# Commande exécutée :  
../.venv/bin/pytest tests/ -v -m "not performance"
```

Explication du paramètre -m "not performance" :

- **-m** : Sélectionne les tests par marker
- **"not performance"** : Exclut tous les tests marqués avec `@pytest.mark.performance`

Sortie attendue :

```
===== test session starts =====
collected 82 items / 6 deselected / 76 selected

tests/test_api.py::test_triangulation_success PASSED [ 1%]
...
tests/test_unit.py::test_delaunay_euler_formula PASSED [100%]

===== 76 passed, 6 deselected, 6 warnings in 0.30s =====
```

Quand l'utiliser :

- Développement rapide (les tests de performance sont plus lents)
- Validation fonctionnelle sans mesurer les performances
- Pendant le développement itératif

4.3 Commande : `make perf_test`

Description : Lance UNIQUEMENT les tests de performance

Utilisation :

```
make perf_test
```

Détails :

```
# Commande exécutée :
../.venv/bin/pytest tests/ -v -m "performance"
```

Explication du paramètre `-m "performance"` :

- Sélectionne uniquement les tests marqués avec `@pytest.mark.performance`

Tests de performance inclus :

1. `test_delaunay_performance_large_set` : Triangulation de 50 points (< 2s)
2. `test_performance_pointset_to_bytes` : Conversion 1000 points → binaire (< 0.1s)
3. `test_performance_bytes_to_pointset` : Conversion binaire → 1000 points (< 0.1s)
4. `test_performance_triangles_to_bytes` : Conversion 100 points triangulés → binaire (< 0.2s)
5. `test_performance_bytes_to_triangles` : Conversion binaire → triangles (< 0.2s)
6. `test_performance_round_trip_conversion` : Conversion aller-retour (< 0.3s)

Sortie attendue :

```
===== test session starts =====
collected 82 items / 76 deselected / 6 selected

tests/test_performance.py::test_delaunay_performance_large_set PASSED [ 16%]
tests/test_performance.py::test_performance_pointset_to_bytes PASSED [ 33%]
tests/test_performance.py::test_performance_bytes_to_pointset PASSED [ 50%]
tests/test_performance.py::test_performance_triangles_to_bytes PASSED [ 66%]
tests/test_performance.py::test_performance_bytes_to_triangles PASSED [ 83%]
tests/test_performance.py::test_performance_round_trip_conversion PASSED [100%]

===== 6 passed, 76 deselected, 6 warnings in 0.13s =====
```

Quand l'utiliser :

- Optimisation des performances
- Benchmarking après des modifications
- Détection de régressions de performance

4.4 Commande : `make coverage`

Description : Génère un rapport de couverture de code (tests → code source)

Utilisation :

```
make coverage
```

Détails :

```
# Commandes exécutées :  
../.venv/bin/coverage run -m pytest tests/  
../.venv/bin/coverage report  
../.venv/bin/coverage html
```

Explication des étapes :

1. **coverage run -m pytest tests/**
 - Exécute tous les tests en traçant le code couvert
 - Génère un fichier **.coverage** (base de données SQLite)
2. **coverage report**
 - Affiche le rapport de couverture dans le terminal
3. **coverage html**
 - Génère un rapport HTML détaillé dans **htmlcov/**

Sortie attendue :

```
===== test session starts =====  
...  
===== 82 passed, 6 warnings in 0.35s =====  
  
Name                               Stmts   Miss  Cover  
-----  
triangulator/__init__.py           0      0   100%  
triangulator/api.py                42      0   100%  
triangulator/core.py              151      0   100%  
triangulator/utils.py              15      0   100%  
-----  
TOTAL                             208      0   100%  
  
Wrote HTML report to htmlcov/index.html
```

Note : Le module **visualizer.py** est exclu de la couverture (configuré dans **.coveragerc**) car il s'agit d'un module de visualisation difficile à tester automatiquement.

Interprétation des résultats :

- **Stmts** : Nombre total de lignes de code
- **Miss** : Lignes non couvertes par les tests
- **Cover** : Pourcentage de couverture

Visualiser le rapport HTML :

```
open htmlcov/index.html
```

Le rapport HTML permet de :

- Voir ligne par ligne quel code est couvert (vert) ou non (rouge)
- Identifier les branches non testées
- Analyser les cas limites manquants

Quand l'utiliser :

- Après avoir ajouté de nouveaux tests
- Pour identifier les zones non testées
- Avant une release pour assurer une couverture > 90%

4.5 Commande : `make lint`

Description : Valide la qualité du code avec **ruff** (PEP8, documentation, etc.)

Utilisation :

```
make lint
```

Détails :

```
# Commande exécutée :  
../.venv/bin/ruff check triangulator/
```

Règles vérifiées (configurées dans **pyproject.toml**) :

- **E** : Erreurs de style (PEP8)
- **F** : Erreurs logiques (variables non utilisées, imports manquants)
- **D** : Documentation (docstrings manquantes ou mal formatées)
- **N** : Conventions de nommage
- **UP** : Suggestions d'upgrade Python

Sortie attendue (succès) :

```
All checks passed!  
warning: `incorrect-blank-line-before-class` (D203) and `no-blank-line-before-class` (D...
```

Sortie en cas d'erreur :

```
triangulator/core.py:247:1: D401 First line of docstring should be in imperative mood  
triangulator/core.py:256:89: E501 Line too long (120 > 88)  
Found 2 errors.
```

Corriger automatiquement certaines erreurs :

```
ruff check --fix triangulator/
```

Quand l'utiliser :

- Avant chaque commit
- En pre-commit hook
- En CI/CD pour bloquer le code non conforme

4.6 Commande : `make doc`

Description : Génère la documentation HTML automatiquement avec **pdoc3**

Utilisation :

```
make doc
```

Détails :

```
# Commande exécutée :  
../.venv/bin/pdoc3 --html --output-dir docs triangulator/ --force
```

Explication des paramètres :

- `--html` : Génère de la documentation HTML (vs texte brut)
- `--output-dir docs` : Dossier de sortie
- `triangulator/` : Package à documenter
- `--force` : Écrase les fichiers existants

Sortie attendue :

```
docs/triangulator/index.html
docs/triangulator/api.html
docs/triangulator/core.html
docs/triangulator/utils.html
docs/triangulator/visualizer.html
```

Visualiser la documentation :

```
open docs/triangulator/index.html
```

Contenu généré :

- Index du package avec tous les modules
- Documentation de chaque fonction/classe avec :
 - Signature
 - Docstring
 - Paramètres et types
 - Valeurs de retour
 - Exceptions levées

Quand l'utiliser :

- Après avoir modifié des docstrings
- Avant une release pour partager la documentation
- Pour vérifier que toutes les fonctions sont documentées

4.7 Commande : `make doc_pdf`

Description : Génère la documentation au format PDF

Utilisation :

```
make doc_pdf
```

Détails :

```
# Commande exécutée :
../.venv/bin/python generate_pdf_simple.py
```

Sortie attendue :

```
■ Lecture de docs/DOCUMENTATION.md...
■ Création du PDF...
■ PDF généré avec succès : docs/documentation.pdf
■ Taille du fichier : 33.8 Ko
```

Quand l'utiliser :

- Pour générer une version PDF de la documentation
- Avant une release pour distribuer la documentation
- Pour archivage ou impression

4.8 Commande : `make clean`

Description : Nettoie tous les fichiers générés (cache, rapports, etc.)

Utilisation :

```
make clean
```

Détails :

```
# Commandes exécutées :
rm -rf htmlcov/          # Rapports de couverture HTML
rm -rf docs/             # Documentation générée
rm -rf .coverage         # Base de données coverage
rm -rf .pytest_cache/    # Cache pytest
find . -type d -name __pycache__ -exec rm -rf {} + # Fichiers .pyc
find . -type f -name "*.pyc" -delete
```

Quand l'utiliser :

- Avant de régénérer tous les rapports
- Pour libérer de l'espace disque
- Avant un commit pour éviter de versionner des fichiers générés

5. Tests

5.1 Organisation des tests

Les tests sont organisés en 4 fichiers selon leur niveau :

*****tests/test_unit.py** (44 tests)***

Tests unitaires de l'algorithme de triangulation et des conversions binaires.

*****tests/test_performance.py** (6 tests)***

Tests de performance pour mesurer les temps d'exécution :

- Triangulation sur grands ensembles (50 points)
- Performance des conversions binaires (1000 points)
- Conversion aller-retour complète
- Cas limites (points colinéaires, NaN, Inf, etc.)
- Tests spécifiques Delaunay (propriétés du cercle circonscrit, formule d'Euler, etc.)
- Tests de performance (marqués `@pytest.mark.performance`)

*****tests/test_api.py** (3 tests)***

Tests de l'API Flask avec mocking du PointSetManager :

- `test_triangulation_success` : Cas nominal
- `test_triangulation_not_found` : PointSet inexistant (404)

- `test_triangulation_bad_request` : UUID invalide (400)

*****tests/test_integration.py** (7 tests)***

Tests d'intégration du workflow complet :

- `test_integration_full_flow_success` : Workflow complet réussi
- `test_integration_pointset_not_found` : Gestion du 404
- `test_integration_pointset_manager_error` : Gestion du 500
- `test_integration_pointset_manager_timeout` : Gestion du timeout
- `test_integration_invalid_uuid` : UUID malformé
- `test_integration_triangulation_flow` : Vérification du flux
- `test_integration_end_to_end_with_real_data` : Test E2E avec données réelles

*****tests/test_error_handlers.py** (22 tests)***

Tests des gestionnaires d'erreurs et cas exceptionnels :

- Validation des UUIDs invalides
- Gestion des erreurs réseau
- Timeouts et erreurs de connexion
- Erreurs de format binaire
- Gestion des erreurs internes du serveur

5.2 Exécuter un test spécifique

Pour cibler des tests précis :

```
# Un seul test
pytest tests/test_unit.py::test_triangulate_three_points -v

# Tous les tests d'un fichier
pytest tests/test_api.py -v

# Tests correspondant à un pattern
pytest tests/ -k "delaunay" -v

# Tous les tests de performance
pytest tests/test_performance.py -v
```

5.3 Options pytest utiles

Options utiles lors de l'exécution des tests :

```
# Mode verbeux avec détails
pytest tests/ -v

# Afficher les print() dans les tests
pytest tests/ -s

# Arrêter au premier échec
pytest tests/ -x

# Afficher les 10 tests les plus lents
pytest tests/ --durations=10

# Exécuter les tests en parallèle (nécessite pytest-xdist)
pytest tests/ -n auto
```

5.4 Fixtures pytest

Les tests utilisent des fixtures pour le mocking et la configuration :

```
@pytest.fixture
def client():
    """Client Flask pour les tests d'API"""
    app = create_app()
    app.config['TESTING'] = True
    with app.test_client() as client:
        yield client

@pytest.fixture
def mock_pointset_manager_success():
    """Mock d'une réponse réussie du PointSetManager"""
    with patch('TP.triangulator.api.requests.get') as mock_get:
        mock_response = requests.models.Response()
        mock_response.status_code = 200
        mock_response._content = sample_points_binary
        mock_get.return_value = mock_response
        yield mock_get
```

5.5 Markers pytest

Les tests utilisent des markers pour la catégorisation :

```
@pytest.mark.performance
def test_delaunay_performance_large_set():
    """Test de performance avec 50 points"""
    ...
```

Configuration des markers (dans `pyproject.toml`) :

```
[tool.pytest.ini_options]
markers = [
    "performance: marks tests as performance tests (deselect with '-m \"not performance...\"')
]
```

6. Couverture de code

6.1 Objectif de couverture

Le projet atteint une couverture de **100%** des modules principaux (api, core, utils).

6.2 Analyser la couverture

Commandes pour analyser la couverture de code :

```
# Générer et afficher le rapport
make coverage
```

```
# Voir uniquement les lignes manquantes
coverage report --show-missing

# Rapport détaillé d'un fichier spécifique
coverage report --include="triangulator/core.py"
```

6.3 Rapport HTML interactif

Pour visualiser la couverture de manière interactive :

```
# Générer et ouvrir le rapport
make coverage
open htmlcov/index.html
```

Le rapport HTML permet de :

- Cliquer sur chaque fichier pour voir les lignes couvertes/non couvertes
- Identifier les branches conditionnelles non testées
- Naviguer facilement dans le code source

6.4 Configuration de coverage

Fichier **.coveragerc** :

```
[run]
omit =
    triangulator/visualizer.py
    */tests/*
    */__pycache__/*
    */venv/*
    */.venv/*

[report]
exclude_lines =
    pragma: no cover
    def __repr__
    raise AssertionError
    raise NotImplementedError
    if __name__ == '__main__':
    if TYPE_CHECKING:
    @abstract

[html]
directory = htmlcov
```

Pourquoi exclure **visualizer.py ?**

- Module de visualisation graphique difficile à tester automatiquement
- Nécessite des dépendances graphiques (matplotlib, imageio)
- Tests manuels via les exemples dans **examples/**

7. Qualité du code

7.1 Règles ruff

Le fichier `pyproject.toml` définit les règles de qualité :

```
[tool.ruff]
line-length = 88
target-version = "py310"

[tool.ruff.lint]
select = [
    "E", # pycodestyle errors
    "W", # pycodestyle warnings
    "F", # pyflakes
    "D", # pydocstyle (documentation)
    "N", # pep8-naming
    "UP", # pyupgrade
]

[tool.ruff.lint.pydocstyle]
convention = "google"
```

7.2 Vérifier la qualité

Pour vérifier et améliorer la qualité du code :

```
# Vérifier sans corriger
make lint

# Corriger automatiquement ce qui peut l'être
ruff check --fix triangulator/

# Formater le code
ruff format triangulator/
```

7.3 Ignorer certaines règles ponctuellement

Dans certains cas, vous pouvez ignorer des règles spécifiques :

```
# Ignorer une règle sur une ligne
long_variable_name = "value" # noqa: E501

# Ignorer une règle sur un fichier
# ruff: noqa: D100
```

7.4 Intégration avec l'éditeur

VS Code (`.vscode/settings.json`) :

```
{
    "python.linting.enabled": true,
    "python.linting.ruffEnabled": true,
    "editor.formatOnSave": true,
    "python.formatting.provider": "ruff"
}
```

8. Documentation automatique

8.1 Générer la documentation

Pour générer la documentation HTML à partir des docstrings :

```
make doc
```

La documentation sera générée dans `docs/triangulator/`.

8.2 Format des docstrings

Le projet utilise le **format Google** :

```
def triangulate(points):
    """Effectue une triangulation de Delaunay d'un ensemble de points 2D.

    La triangulation de Delaunay maximise l'angle minimal de tous les triangles,
    ce qui évite les triangles "plats" et produit une triangulation optimale.

    Algorithme: Bowyer-Watson incrémental

    Args:
        points: Liste de tuples (x, y) représentant les coordonnées des points.

    Returns:
        Liste de triangles, où chaque triangle est un tuple de 3 indices.

    Raises:
        ValueError: Si moins de 3 points, ou si les points sont invalides.
        TypeError: Si les coordonnées ne sont pas des nombres.

    """
```

8.3 Visualiser la documentation

```
# Ouvrir dans le navigateur
open docs/triangulator/index.html

# Ou servir avec un serveur HTTP
python -m http.server 8000 --directory docs/
# Puis ouvrir http://localhost:8000/triangulator/
```

9. Algorithme de triangulation

9.1 Triangulation de Delaunay

1. Crée un **super-triangle** englobant tous les points
2. Ajoute chaque point un par un
3. Pour chaque nouveau point :
 - Trouve les triangles dont le cercle circonscrit contient le point
 - Supprime ces triangles (appelés "bad triangles")
 - Trouve les arêtes du polygone formé
 - Crée de nouveaux triangles avec le point et ces arêtes
4. Supprime les triangles contenant les sommets du super-triangle

```
00 00 00 03    # 3 points
00 00 00 00    # Point 0 X = 0.0
```

```
00 00 00 00 # Point 0 Y = 0.0
3f 80 00 00 # Point 1 X = 1.0
00 00 00 00 # Point 1 Y = 0.0
00 00 00 00 # Point 2 X = 0.0
3f 80 00 00 # Point 2 Y = 1.0
```

10.2 Format Triangles

```

■ PointSet (format ci-dessus) ■
■ Nombre de triangles (4 bytes, unsigned int) ■
■ Triangle 0 - Indice A (4 bytes, unsigned int) ■
■ Triangle 0 - Indice B (4 bytes, unsigned int) ■
■ Triangle 0 - Indice C (4 bytes, unsigned int) ■
■ Triangle 1 - Indice A (4 bytes, unsigned int) ■
■ Triangle 1 - Indice B (4 bytes, unsigned int) ■
■ Triangle 1 - Indice C (4 bytes, unsigned int) ■
■ ... ■

```

10.3 Conversion en Python

```
import struct

# PointSet → bytes
def pointset_to_bytes(points):
    n = len(points)
    binary = struct.pack(">I", n) # ">I" = big-endian unsigned int
    for x, y in points:
        binary += struct.pack(">ff", x, y) # ">ff" = 2 floats big-endian
    return binary

# bytes → PointSet
def bytes_to_pointset(binary):
    n = struct.unpack(">I", binary[:4])[0]
    points = []
    offset = 4
    for _ in range(n):
        x, y = struct.unpack(">ff", binary[offset:offset+8])
        points.append((x, y))
        offset += 8
    return points
```

11. API REST

11.1 Endpoint de triangulation

Route : GET /triangulation/

Description : Récupère un PointSet depuis le PointSetManager et retourne sa triangulation

Paramètres :

- **pointset_id** (path) : UUID du PointSet à trianguler

Réponses :**200 OK - Succès**

Content-Type: application/octet-stream
Body: <données binaires au format Triangles>

400 Bad Request - UUID invalide

```
{
  "code": "INVALID_UUID",
  "message": "The provided PointSetID is not a valid UUID"
}
```

404 Not Found - PointSet inexistant

```
{
  "code": "POINT_SET_NOT_FOUND",
  "message": "PointSet with ID <uuid> not found"
}
```

500 Internal Server Error - Erreur serveur

```
{
  "code": "INTERNAL_ERROR",
  "message": "An internal error occurred"
}
```

503 Service Unavailable - PointSetManager inaccessible

```
{
  "code": "SERVICE_UNAVAILABLE",
  "message": "PointSetManager is not available"
}
```

11.2 Exemple d'utilisation

```
# Avec curl
curl -X GET http://localhost:5000/triangulation/123e4567-e89b-12d3-a456-426614174000 \
  --output triangles.bin

# Avec Python requests
import requests

pointset_id = "123e4567-e89b-12d3-a456-426614174000"
response = requests.get(f"http://localhost:5000/triangulation/{pointset_id}")

if response.status_code == 200:
    triangles_binary = response.content
    # Décoder les triangles...
else:
    error = response.json()
    print(f"Erreur: {error['message']}")
```

11.3 Lancer le serveur

```
# Mode développement
python -m triangulator.api

# Mode production (avec gunicorn)
gunicorn triangulator.api:app -b 0.0.0.0:5000 -w 4
```

12. Dépannage

12.1 Problèmes courants

*****Erreur : `ModuleNotFoundError: No module named 'TP'`*****

Cause : PYTHONPATH n'est pas configuré

Solution :

```
export PYTHONPATH=$(pwd)/..
# Ou utiliser les commandes make qui le configurent automatiquement
```

*****Erreur : `pytest: command not found`*****

Cause : Environnement virtuel non activé ou pytest non installé

Solution :

```
source ../.venv/bin/activate
pip install -r dev_requirements.txt
```

*****Tests échouent avec des erreurs de format binaire*****

Cause : Problème d'endianness (little-endian vs big-endian)

Solution : Vérifier que `struct.pack(">I")` est bien utilisé (big-endian)

*****Coverage à 0%*****

Cause : Les tests ne sont pas exécutés via coverage

Solution :

```
# Utiliser make coverage, PAS pytest directement
make coverage
```

*****Ruff trouve des erreurs de documentation*****

Cause : Docstrings manquantes ou mal formatées

Solution :

```
# Voir les erreurs
make lint

# Exemple de docstring correcte
def ma_fonction(param1, param2):
    """Brève description de la fonction.

    Description détaillée optionnelle.

    Args:
        param1: Description du paramètre 1.
        param2: Description du paramètre 2.

    Returns:
        Description de la valeur de retour.

    Raises:
        ValueError: Quand la valeur est invalide.
    """
```

12.2 Debug des tests

```
# Afficher les print() dans les tests
pytest tests/ -s

# Mode verbeux avec traceback complet
pytest tests/ -vv

# Arrêter au premier échec
pytest tests/ -x

# Lancer le debugger sur échec
pytest tests/ --pdb
```

12.3 Performances lentes

```
# Identifier les tests lents
pytest tests/ --durations=10

# Exclure les tests de performance
make unit_test

# Profiler le code
python -m cProfile -o profile.stats triangulator/core.py
python -m pstats profile.stats
```

Annexes

A. Résumé des commandes

B. Métriques du projet

- **Lignes de code** : ~1200 (source + tests + visualisation)
- **Couverture** : 100% (modules principaux)
- **Tests** : 82 (51 unitaires, 7 intégration, 3 API, 22 erreurs, 6 performance)
- **Conformité ruff** : 100% (0 erreurs)
- **Documentation** : 100% (toutes les fonctions publiques)
- **Modules** : 5 (api, core, utils, visualizer, __init__)

C. Ressources utiles

- **pytest** : <https://docs.pytest.org/>
- **coverage** : <https://coverage.readthedocs.io/>
- **ruff** : <https://docs.astral.sh/ruff/>
- **pdoc3** : <https://pdoc3.github.io/pdoc/>
- **Delaunay** : https://en.wikipedia.org/wiki/Delaunay_triangulation
- **Bowyer-Watson** : https://en.wikipedia.org/wiki/Bowyer%E2%80%93Watson_algorithm

Dernière mise à jour : Décembre 2025

Version : 1.0.0

Auteur : Équipe Triangulator